6. Index Usage with JOINs

Objective

This session explains how PostgreSQL executes JOIN operations and how indexes influence their performance.

After this session, you should understand:

- how PostgreSQL performs different JOIN algorithms
- how indexes are used during JOINs
- how indexes affect each JOIN algorithm differently
- how PostgreSQL chooses the most efficient JOIN algorithm
- why indexing join keys is one of the most important database optimization techniques

1. Introduction

A JOIN combines rows from two tables based on a matching condition.

Example:

```
SELECT *
FROM customers c
JOIN orders o
    ON c.id = o.customer_id;
```

The database needs to find every order that belongs to every customer. Conceptually, this sounds simple, but the execution strategy can vary enormously. For example, if `customers` contains 1,000 rows and `orders` contains 100 million rows, checking every order for every customer would require approximately **100 billion comparisons** (`1,000 × 100,000,000`), which is clearly impractical.

Instead, PostgreSQL chooses one of three physical JOIN algorithms:

- **Nested Loop Join**
- **Hash Join**
- **Merge Join**

The choice of algorithm often has a much larger impact on query performance than the SQL query itself.

2. Nested Loop Join

Nested Loop is the simplest JOIN algorithm. It iterates over one table (the **outer table**) and, for each row, searches the second table (the **inner table**) for matching rows.

Consider the following tables:

Customers

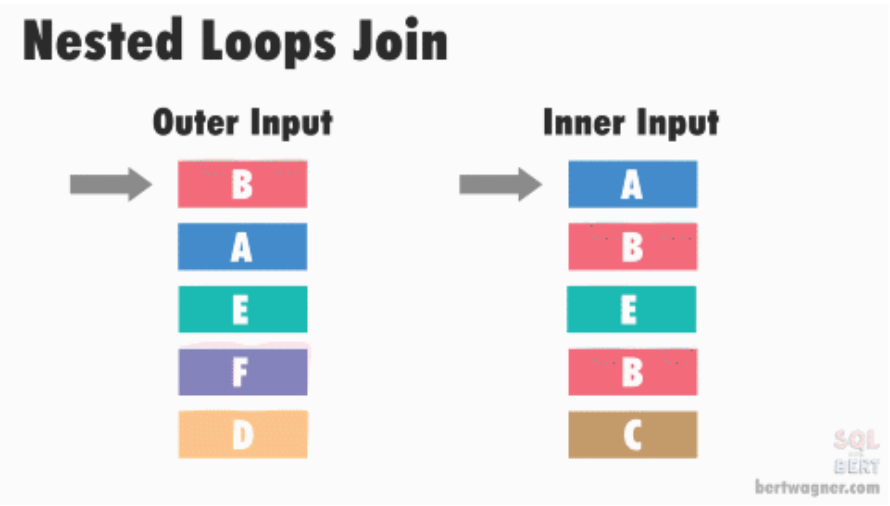
id	name
1	Alice
2	Bob
3	Charlie

Orders

id	customer_id
10	2
11	1
12	1

Conceptually, PostgreSQL performs:

```
For each customer
    Search the orders table
        If customer.id = order.customer_id
            Return the matching row
```



Visual representation

Without indexes, each customer may require scanning the entire `orders` table. If the outer table contains **N** rows and the inner table contains **M** rows, the algorithm may perform up to `N × M` comparisons.

This approach is acceptable only when one of the tables is relatively small.

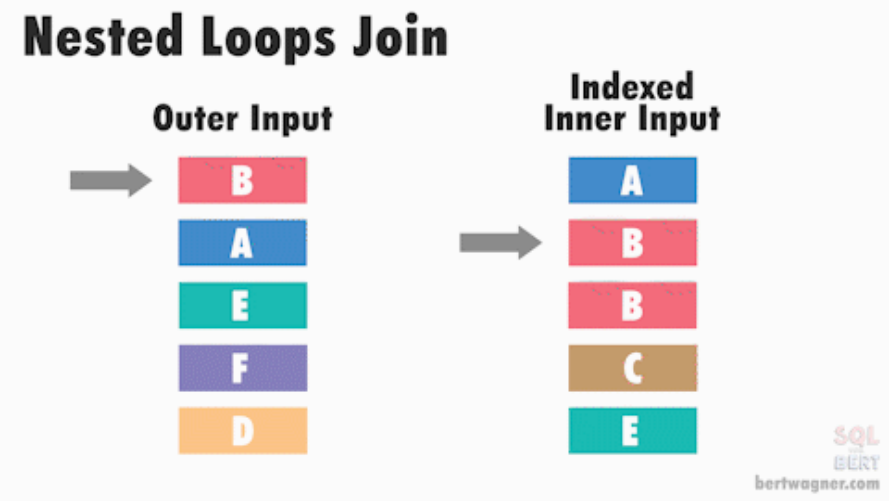
Nested Loop with an Index

Now suppose we create an index:

```
CREATE INDEX idx_orders_customer
ON orders(customer_id);
```

Instead of scanning the entire `orders` table for every customer, PostgreSQL performs a B-tree lookup on `customer_id` and immediately locates the matching rows.

Instead of repeatedly scanning millions of rows, each lookup requires only a logarithmic search through the index. The complexity becomes approximately `N × log(M)`, making Nested Loop extremely efficient when the outer table is small and the inner table is indexed.



Visual representation

Advantages

- Excellent when the outer table is small.
- Very efficient if the inner table has an index on the join key.
- Can start returning rows immediately without processing the entire input.
- Requires very little memory.

Disadvantages

- Without an appropriate index, the inner table is scanned repeatedly, making performance degrade rapidly as table sizes grow.
- Usually not suitable when both tables are large.

3. Hash Join

Hash Join is designed specifically for **equality joins**, such as:

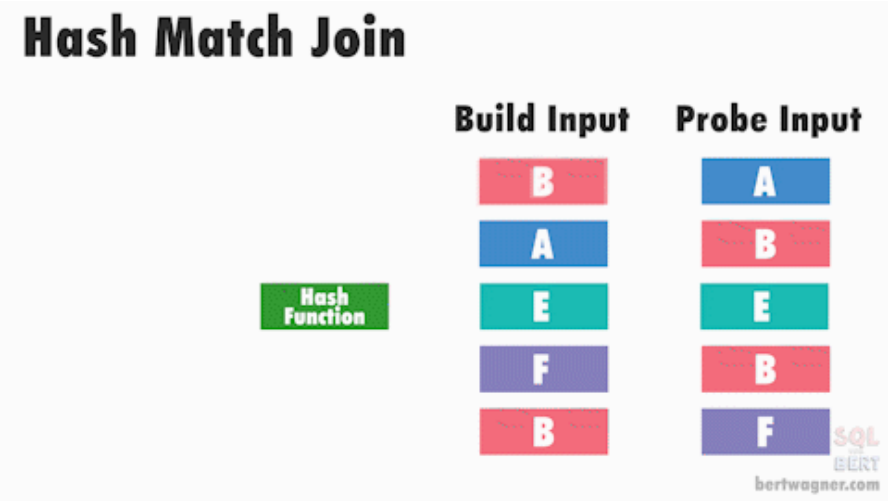
```
ON c.id = o.customer_id
```

Instead of repeatedly searching one table, PostgreSQL builds an in-memory hash table from the smaller input.

For example, the `customers` table might be transformed into:

Hash Key	Customer
1	Alice
2	Bob
3	Charlie

PostgreSQL then scans the `orders` table only once. For every order, it computes the hash value of `customer_id` and performs a constant-time lookup in the hash table to find the matching customer.



Visual representation

Building the hash table requires scanning one table once, while probing it requires scanning the second table once. Therefore, the overall complexity is approximately `N + M`.

Advantages

- Excellent for joining large tables.
- Does not require indexes on the join keys.
- Usually the fastest option for large equality joins.
- Reads each table only once.

Disadvantages

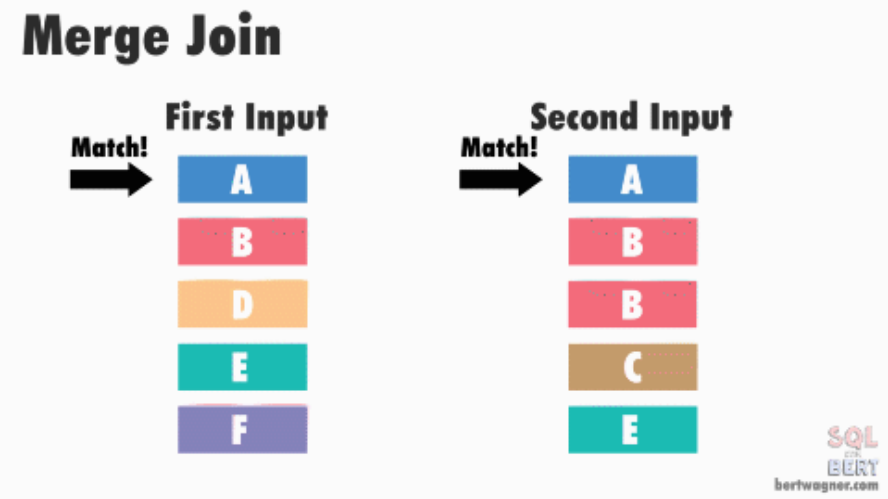
- Works only for equality conditions (=).
- Requires sufficient memory (`work_mem`) to hold the hash table.
- If the hash table does not fit in memory, PostgreSQL spills it to disk, which can significantly increase execution time.

4. Merge Join

Merge Join requires both inputs to be sorted on the join key.

Suppose the tables are already ordered, Merge Join walks through both inputs simultaneously, much like merging two sorted lists.

- If the keys match, PostgreSQL returns the joined row.
- If one key is smaller, only that input advances.
- No rows are revisited



Visual representation

Since each row is processed only once, the complexity is approximately `N + M` when the inputs are already sorted.

If sorting is required beforehand, the overall cost becomes roughly `N log N + M log M`.

Advantages

- Excellent for joining very large tables.
- Reads each input only once.
- Supports ordered comparisons in addition to equality joins.
- Particularly efficient when the data is already sorted.

Disadvantages

- Both inputs must be sorted.
- Sorting large tables can be expensive if suitable indexes do not exist.

5. How Indexes Affect Each JOIN Algorithm

Indexes do not benefit every JOIN algorithm equally.

Nested Loop Join

Nested Loop benefits the most from indexes.

Without an index, PostgreSQL scans the entire inner table for every row in the outer table.

With an index on the join key, PostgreSQL performs a fast B-tree lookup for each outer row, avoiding repeated table scans.

This is the classic scenario where indexes provide dramatic performance improvements.

Hash Join

Indexes generally do **not** help the join itself.

Hash Join performs sequential scans because every row must be read anyway to build or probe the hash table. Reading the table sequentially is typically faster than performing millions of random index lookups.

However, indexes may still improve performance **before** the join.

For example:

```
SELECT *
FROM customers c
JOIN orders o
    ON c.id = o.customer_id
WHERE o.status = 'ACTIVE';
```

If only 1% of orders are active, an index on `status` allows PostgreSQL to retrieve only those rows before constructing the hash table, significantly reducing the amount of data involved in the join.

Merge Join

Indexes can eliminate the expensive sorting phase.

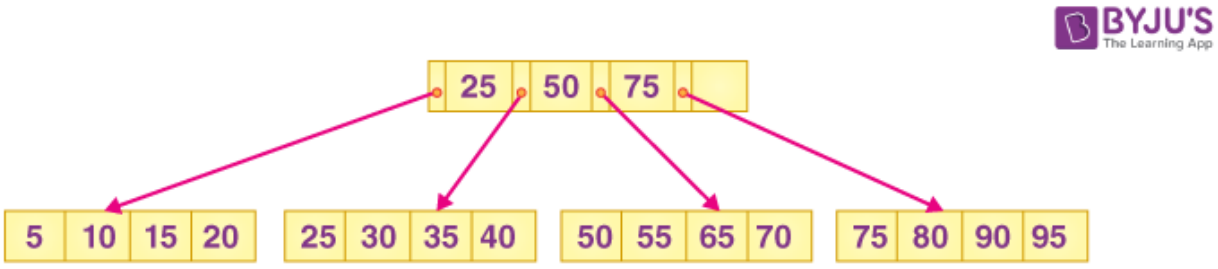
B-tree indexes naturally return rows in sorted order.

For example:

```
CREATE INDEX idx_customers_id
ON customers(id);
```

```
CREATE INDEX idx_orders_customer
ON orders(customer_id);
```

Instead of performing explicit sort operations, PostgreSQL may perform index scans on both tables and feed the already sorted rows directly into the Merge Join.



B-plus Tree visualization

This can substantially reduce execution time for large joins.

6. Example

Suppose:

- `customers` contains **1 million rows**
- `orders` contains **50 million rows**

Query:

```
SELECT *
FROM customers c
JOIN orders o
ON c.id = o.customer_id;
```

Scenario 1 — No indexes

A likely execution plan is:

```
Hash Join
  -> Seq Scan customers
  -> Seq Scan orders
```

Since both tables must be read entirely, sequential scans combined with a Hash Join are usually the most efficient strategy.

Scenario 2 — Index on `orders(customer_id)`

A possible plan becomes:

```
Nested Loop
  -> Seq Scan customers
  -> Index Scan orders
```

Each customer performs a fast index lookup to retrieve matching orders.

Scenario 3 — Indexes on both join keys

If both `customers(id)` and `orders(customer_id)` are indexed, PostgreSQL may choose:

```
Merge Join
  -> Index Scan customers
  -> Index Scan orders
```

Because both index scans produce sorted output, no explicit sorting is required.

7. How PostgreSQL Chooses a JOIN Algorithm

PostgreSQL uses a **cost-based optimizer**.

Rather than selecting a JOIN algorithm based solely on whether indexes exist, it estimates the execution cost of many possible plans and chooses the one with the lowest estimated cost.

The optimizer considers several factors.

Table Statistics

PostgreSQL maintains statistics about tables through `ANALYZE` (automatically executed by autovacuum or manually).

These statistics include:

- number of rows
- number of pages
- number of distinct values
- value distribution (histograms)
- most common values
- percentage of NULL values
- correlation between data and physical storage

These statistics allow PostgreSQL to estimate how many rows will participate in each step of the query.

For example:

```
SELECT *
FROM customers c
JOIN orders o
    ON c.id = o.customer_id
WHERE c.country = 'AZ';
```

If PostgreSQL knows only 2% of customers are located in Azerbaijan, it estimates that only a small fraction of rows will participate in the join. This may make a Nested Loop Join with index lookups significantly cheaper than building a hash table over millions of rows.

Outdated statistics can result in poor execution plans, which is why keeping statistics up to date is important.

Estimated Number of Rows

Before choosing a JOIN algorithm, PostgreSQL estimates how many rows remain after filtering.

A predicate such as this may reduce a 100-million-row table to only a few hundred rows, dramatically changing the optimal strategy:

```
WHERE customer_id = 15
```

Available Indexes

The optimizer evaluates questions such as:

- Can an index avoid scanning the entire table?
- Can an index provide rows in sorted order?
- Is using the index cheaper than a sequential scan?

Indexes reduce cost only when PostgreSQL estimates that using them is actually faster.

Join Condition

The join predicate itself limits which algorithms are available.

Join Condition	Nested Loop	Hash Join	Merge Join
=			
<, <=, >, >=			

Sorting Cost

Merge Join requires sorted inputs.

If sorting both tables is expensive, PostgreSQL may instead choose a Hash Join. However, if indexes already provide sorted data, Merge Join becomes much more attractive.

Memory

Hash Join depends heavily on available memory.

If PostgreSQL estimates that the hash table will exceed `work_mem`, it anticipates disk spilling, increasing the estimated cost of the Hash Join.

Overall Decision

The optimizer estimates the cost of every candidate plan.

For example:

Join Algorithm	Estimated Cost
Nested Loop	150
Merge Join	280
Hash Join	320

In this case, PostgreSQL selects the Nested Loop Join because it has the lowest estimated cost.

You can inspect these estimates using:

```
EXPLAIN
SELECT ...;
```

or compare them with actual execution statistics using:

```
EXPLAIN ANALYZE
SELECT ...;
```

8. Importance of Indexing Join Keys

Join keys are among the most valuable columns to index because they directly affect how efficiently tables can be combined.

For example: `customers.id` joining with `orders.customer_id`

allows PostgreSQL to locate matching orders immediately instead of repeatedly scanning the `orders` table. Indexes on join keys also enable Merge Joins by providing rows in sorted order, often eliminating expensive sort operations.

Indexes become even more valuable when joins are combined with filtering.

For example:

```
SELECT *
FROM customers c
JOIN orders o
    ON c.id = o.customer_id
WHERE o.status = 'OPEN';
```

An index such as this can both filter the rows by `status` and provide efficient access to the join key, reducing the amount of data involved in the join:

```
CREATE INDEX idx_orders_status_customer
ON orders(status, customer_id);
```

However, indexes also have costs:

- They consume additional disk space.
- Every `INSERT`, `UPDATE`, and `DELETE` must update the index.
- Too many indexes can negatively affect write performance.

As a result, join keys—particularly foreign keys and frequently joined columns—are often among the highest-priority candidates for indexing.

Key Takeaways

- PostgreSQL supports three primary JOIN algorithms: **Nested Loop**, **Hash Join**, and **Merge Join**.
- Nested Loop performs best when the outer table is small and the inner table has an index on the join key.
- Hash Join is usually the fastest option for large equality joins and generally does not benefit from indexes during the join itself.
- Merge Join is highly efficient when both inputs are already sorted, often because B-tree indexes provide rows in join-key order.
- PostgreSQL uses a cost-based optimizer that considers statistics, row estimates, indexes, sorting costs, memory requirements, and I/O costs before selecting a JOIN algorithm.
- Accurate table statistics are essential for choosing efficient execution plans.
- Indexing join keys—especially foreign keys—is one of the most effective ways to improve JOIN performance, although indexes should be created thoughtfully due to their maintenance overhead.